

MUNICTECH TECHNICAL JUDGE FAQ

AI Action Path Assurance by HAIEC

This document is designed for the skeptical technical judge. It answers the questions most likely to arise after seeing the live KestrelVoice evaluation.

TECHNICAL ORIENTATION —

1. What is the difference between access and reachability?

Access describes what an identity, policy, or tool is allowed to use. Reachability describes what the connected implementation can actually reach through code, tools, services, resources, and downstream effects.

2. What is the unit of assurance?

The conceptual unit of assurance is the AI Action Path: the qualified route from AI-facing capability to consequence. The AI Action Path Trail is the inspectable evidence representation of that path.

3. Is HAIEC using AI to judge AI?

No. Not AI testing AI. The assurance claim is deterministic and source-backed. HAIEC binds the evaluation to an exact source snapshot, qualifies relationships into canonical evidence, and keeps stronger authority/runtime claims separate.

4. Why keep five evidence planes independent?

Because one kind of evidence must not silently answer another. Code capability is not authorization; authorization is not runtime execution.

5. What is the difference between a declared control and a path-bound control?

A declared control exists in policy, configuration, documentation, or platform state. A path-bound control is established on the exact consequential action path being reviewed.

6. What does REVIEW mean, and how does it move toward ALLOW?

REVIEW means the current evidence establishes consequential capability while one or more assurance questions still require resolution. The team connects the missing intent, policy, effective-authority, path-bound-control, or runtime evidence and re-evaluates.

7. Why does UNRESOLVED ≠ ABSENT?

If the evidence cannot establish that a path or control disappeared, HAIEC does not convert incomplete absence evidence into a removal claim. Unresolved remains unresolved.

8. What makes a Release A / Release B comparison valid?

The evaluations must be safely comparable: same system boundary, compatible completed evaluations, qualified scope, and comparable analyzer conditions. In A/B-8 the release perimeter is SAME, analyzer comparability is EXACT, and repeatability is PASS.

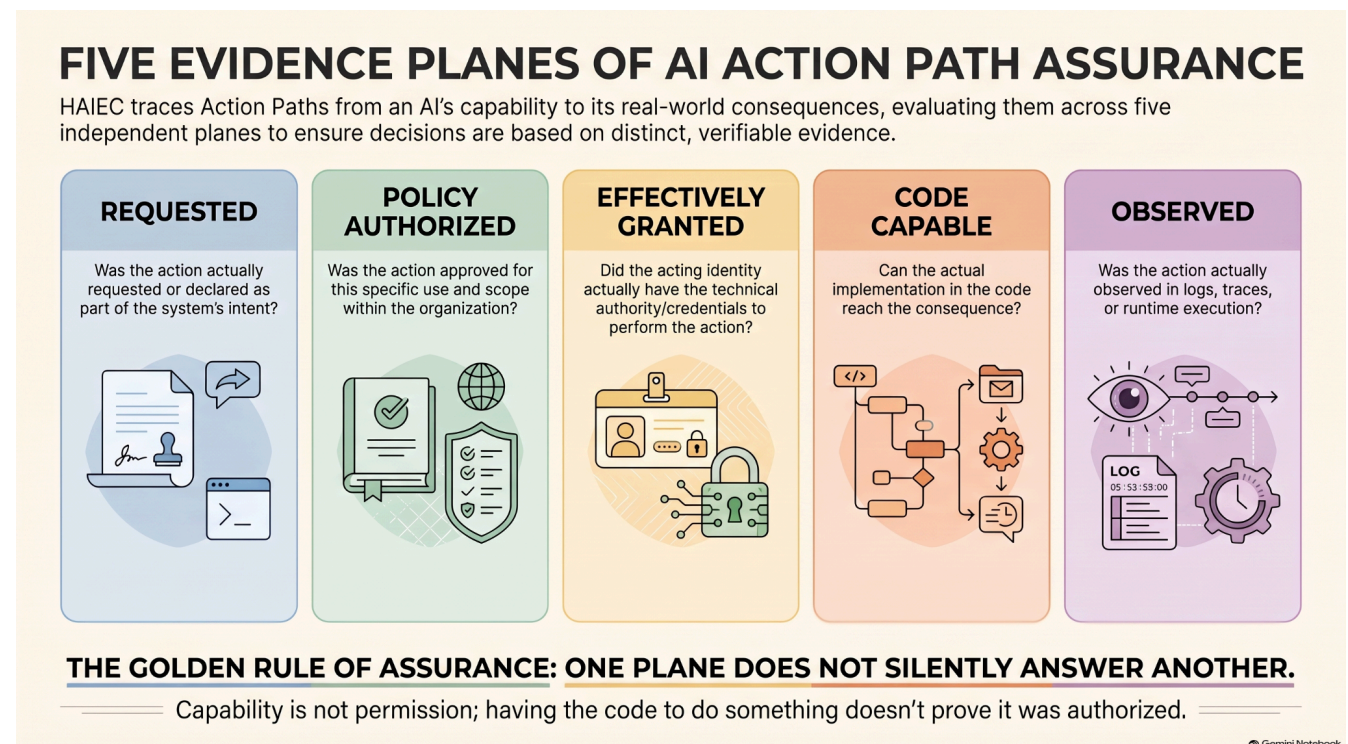
9. Does HAIEC retain customer source code?

The normal workflow requires temporary authorized source access. Source is processed in an ephemeral analysis environment and the source workspace is discarded after analysis. HAIEC retains structured evidence, findings, hashes, metadata, and assurance outputs rather than a persisted repository copy.

10. Can HAIEC support stricter enterprise environments?

Yes, as an enterprise PoC pattern. For organizations that cannot permit external source processing, HAIEC can scope a customer-controlled or private analysis path. Do not interpret this as a claim that a general-availability self-hosted edition is already packaged.

FIVE EVIDENCE PLANES AT A GLANCE



Visual summary: one evidence plane does not silently answer another.

Q1. Is HAIEC just another SAST scanner?

No.

Source and semantic analysis are inputs. AI Action Path Assurance composes qualified relationships into an AI Action Path Trail and then reconciles that capability with independent authority and runtime evidence.

The product asks:

What AI-facing capability exists?

Is it exposed?

What implementation does it reach?

What consequential operation does that implementation reach?

What arguments, identity context, tenant context, controls, services, and resources are on the path?

What authority was requested, authorized, effectively granted, and observed?

A scanner finding is not the final unit of output. The action path, consequence, evidence state, and release decision are.

Q2. Are you asking an LLM to inspect the code and tell you what the agent can do?

No.

Not AI testing AI. The assurance result is deterministic and source-backed. HAIEC binds analysis to an exact source snapshot and qualifies relationships into canonical evidence; an AI model is not the judge of the assurance claim.

An AI model is not the judge of the assurance claim.

Q3. Are you mainly reading README files, Markdown, policy text, or developer declarations?

No.

Documentation or declarations may contribute context to specific evidence questions, but they do not establish the Code Capable action path.

The Code Capable path is established from source-backed relationships such as:

AI-facing capability

→ model exposure / registration

→ dispatch

→ implementation

→ handler

→ consequential operation.

Where evidence exists, HAIEC continues into arguments, authenticated or tenant context, control mediation, resources, services, and persistence.

That is why “a tool exists” is not enough and “a README says it is allowed” is not enough.

Q4. How can HAIEC know what an AI can do without running the AI?

Because Code Capability and Observed execution are intentionally different questions.

Source analysis can establish that an AI-facing path exists in code and reaches a consequential operation.

It cannot automatically prove that the path executed at runtime.

HAIEC keeps those claims separate:

Code Capable = source-backed capability.

Observed = qualifying runtime evidence.

This is a design strength, not a gap. It prevents static evidence from being promoted into a runtime claim.

Q5. What exactly is an AI Action Path Trail?

It is the source-backed chain that connects an AI-facing capability to a consequential system operation.

A simplified trail:

Agent

→ capability

- model exposure
- dispatch
- implementation
- handler
- consequence.

A richer trail may also include:

effective arguments

- authenticated / tenant context
- confirmation or approval mediation
- resource / service reachability
- persistence or other downstream effects.

Q6. What is the Verified Action Frontier?

It is the current verification edge of an AI Action Path Trail.

Plainly:

HAIEC shows how far the action path is verified from the available evidence and identifies the next evidence that would support a stronger assurance conclusion.

It is not a document-reading boundary.

It is not a defect count.

It is not a vulnerability count.

It is a structured assurance output about verification depth.

Q7. Why is the KestrelVoice decision REVIEW if all 44 paths are Code Capable?

Because capability is only one evidence dimension.

For the frozen KestrelVoice evaluation:

All 44 evaluated consequential paths met the Code Capability criteria.

Requested = NOT_ASSESSED.

Policy Authorized = NOT_ASSESSED.

Effectively Granted = NOT_ASSESSED.

Observed = NOT_ASSESSED.

REVIEW means consequential capability is established, while stronger release confidence depends on resolving the relevant authority and evidence questions.

Q8. Does 44 paths mean 44 vulnerabilities?

No.

An action path is evidence that a consequential capability is reachable in code.

Some paths may be entirely intended and appropriate.

Some may require stronger controls.

Some may require better evidence.

Some may become findings only after comparison against policy, scope, or other assurance criteria.

Path count is not risk score.

Q9. Does ORDER RECORD WRITE mean an order was actually placed?

No.

For KestrelVoice, HAIEC established a source-backed path to an orders-table insert.

That supports:

“the evaluated code contains a qualified AI-reachable path to an order-record write.”

It does not by itself support:

“the order was executed at runtime,”

“the organization authorized it,”

“runtime credentials allowed it,” or

“a customer transaction completed.”

Those are separate evidence questions.

Q10. How is HAIEC different from IAM?

IAM answers questions about identity permissions and access policy.

HAIEC asks a different question:

What AI-facing code path can reach a consequential operation, and how does that capability align with intent, policy, effective authority, controls, and observation?

Permission is not delegation.

A permission can exist without an AI path.

An AI path can exist without the evidence needed to establish approved authority.

HAIEC reconciles those domains instead of treating them as interchangeable.

Q11. How is HAIEC different from runtime monitoring?

Runtime monitoring is strongest at showing what happened.

HAIEC is designed to also establish source-backed capability before a runtime event occurs.

The two are complementary:

Static / source-backed evidence: what the code can reach.

Runtime evidence: what actually happened.

HAIEC keeps both available without collapsing one into the other.

Q12. How do you handle enterprise-specific actions that your standard model has never seen?

Standard capability recognition is the default. An enterprise does not need to define every ordinary action before HAIEC can run.

Customization is available for proprietary or domain-specific actions.

Examples:

Telecom: restartCell(), changeAntennaTilt(), publishRAppPolicy(), reconfigureCarrier()
Banking: releaseWireTransfer(), freezeAccount(), adjustCreditLimit()
Manufacturing: restartProductionCell(), changeRobotSpeed(), publishPLCConfiguration()

HAIEC can represent those actions through qualified enterprise capability packs and compare them with requested capabilities and an approved Operating Envelope.

Q13. Does that mean the enterprise tells HAIEC what the consequence is?

Not for standard capabilities.

HAIEC already carries a standard capability ontology and qualified recognition model.

Enterprise input is mainly useful when:

- the operation is proprietary;
- the business meaning is organization-specific;
- the organization wants to define an approved operating boundary;
- domain-specific scope, approval, target, or change limits matter.

The enterprise adds context where its system is unique. It does not create the entire assurance model from scratch.

Q14. What is the Operating Envelope?

The Operating Envelope is the enterprise-approved boundary for consequential AI operation.

It can express:

allowed operations,
resource scopes,
data classes,
destinations,
approved models and tools,
allowed environments,
target-count limits,
change-magnitude limits,
and approval requirements.

That lets HAIEC compare:
what the system can do
versus
what the enterprise intended and approved.

Q15. What is Consequence Delta?

Consequence Delta compares two qualified evaluations of the same AI system.

It is designed to identify changes such as:
added or removed consequences,
expanded or narrowed resource reachability,
service reachability changes,
approval-control changes,
egress changes,
supply-chain changes,
memory/context changes,
deferred-execution changes,
and evidence-coverage changes.

Git tells you what changed in code.
Consequence Delta tells you what changed in what the AI-enabled system can reach and cause.

For this submission, a qualified KestrelVoice Release A / Release B comparison is included. A/B-8 is overall INCONCLUSIVE, with 12 established CONTROL_CHANGED items, 99 UNRESOLVED positional facts, SAME release perimeter, EXACT analyzer comparability, and repeatability PASS. UNRESOLVED ≠ ABSENT.

Q16. Why should a judge trust the published KestrelVoice result?

The public demo is tied to a frozen source snapshot and evaluation.

The evidence package includes multiple projections of the same evaluated evidence, SHA-256 artifact integrity, and a local verifier.

The judge can inspect:

the public report,
the System Consequence Map,
the source-backed path,
the machine-readable evidence,
the Passport,
the Decision Receipt,
and the artifact manifest.

Q17. What did you actually build during MunichTech?

The public competition repository separates the pre-existing HAIEC foundation from the competition-period work.

Competition-period capabilities include the System Consequence Map (Constellation projection), five-plane Action Assurance, Agentic Production Passport integration, Consequence Delta, report projection, artifact/output convergence, and the KestrelVoice public demo experience.

The exact Git provenance is published in ORIGINALITY.md.

Q18. Is HAIEC claiming certification or legal compliance?

No.

HAIEC produces technical assurance evidence and bounded decisions for the evaluated scope.

The Passport is a portable assurance record, not a certification.

Legal or regulatory interpretation remains separate where it requires qualified human judgment.

Q19. What is the core technical insight?

The useful assurance unit is not just the permission, tool, model, or scanner finding.

It is the path from AI-facing capability to consequence, together with the authority and evidence needed to understand whether that consequence is intended, permitted, reachable, controlled, and observed.

Q20. What should I remember after the demo?

Three things:

1. HAIEC shows what consequential AI action paths the code can actually reach.

2. HAIEC keeps capability, authority, and observation separate instead of overstating certainty.

3. HAIEC turns that evidence into a release decision and portable record that can travel with the evaluated system.

What can your AI actually cause?

Q21. How much of the KestrelVoice result was automatic?

The judge-facing answer should be precise:

HAIEC automatically identifies and qualifies supported AI-facing action paths from the evaluated source snapshot. Enterprise input is not required for ordinary supported capability recognition.

Enterprise input becomes useful when the organization has proprietary business actions, internal policy rules, approved operating boundaries, or runtime/authority evidence that only the enterprise can provide.

Do not say every possible path in the repository was proven. Say the evaluated consequential path set was automatically qualified to the supported evidence level and remains bounded by analysis coverage.

Q22. What exactly does 44 / 44 mean?

44 / 44 means that all 44 consequential action paths in the evaluated KestrelVoice path set satisfied the Code Capability evidence requirement.

It does not mean:

- 100% repository coverage,
- 100% analyzer accuracy,

- 44 vulnerabilities,
- or proof that no additional undiscovered path exists.

Use: “44 consequential action paths confirmed in code.”

Q23. How does HAIEC avoid missing path 45?

No credible source-analysis system should claim perfect discovery in every repository. HAIEC's discipline is to separate discovered evidence from coverage evidence. If a construct cannot be qualified, it should be represented as an unresolved analysis surface or coverage boundary rather than silently converted into safety.

Examples that may require coverage handling include dynamic imports, runtime-only tool registration, framework magic, reflection, generated clients, dependency injection, remote MCP configuration, and external configuration not present in the evaluated source snapshot.

Q24. Do you have precision / recall metrics?

For this MunichTech package, the supported claim is an inspectable KestrelVoice evaluation with source-backed paths and a verifiable evidence package. Broader precision/recall benchmarking should be presented only when measured benchmark data exists.

Do not invent numbers. The correct answer is: “This submission demonstrates source-backed inspectability on a frozen real repository. Broader analyzer accuracy benchmarking is part of the validation program and should be reported with measured datasets, not estimated claims.”

Q25. What does the evidence verifier verify?

The verifier validates artifact integrity and consistency of the published evidence package. It checks the packaged outputs, hashes, manifest relationships, decision consistency and expected evidence artifacts.

Do not claim the verifier independently re-runs the full semantic analyzer unless that capability is actually implemented. Say: “The verifier verifies the evidence package. The semantic action paths are inspectable through their source-backed trail.”

Q26. How does REVIEW become ALLOW?

REVIEW means consequential capability is established and specific assurance questions remain to be resolved before stronger release confidence is justified.

To move toward ALLOW, an enterprise provides or connects evidence for the relevant review flags:

- requested intent or capability manifest,

- policy authorization / Operating Envelope,
- effective authority from IAM, credentials, configuration or provider state,
- confirmation or approval controls bound to the path,
- runtime observation where observed execution is required.

HAIEC can then re-evaluate against the new evidence and produce an updated decision.

Q27. Who defines ALLOW, REVIEW and BLOCK?

The decision model should be presented as evidence-bounded and policy-aware. HAIEC provides the assurance structure and default decision semantics; enterprise deployments can align thresholds and approval rules to an approved Operating Envelope, risk policy and customer requirements.

Do not imply a universal certification decision detached from enterprise context.

Q28. Where does evidence for each plane come from?

Requested: product requirements, ticket/change request, requested-capability manifest, workflow intent.

Policy Authorized: Operating Envelope, enterprise policy, change rules, approved workflow constraints.

Effectively Granted: IAM/provider state, credentials, configuration, runtime identity binding, resource scope and environment.

Code Capable: deterministic source-backed AI Action Path Trail.

Observed: qualifying runtime logs, traces, audit events or execution witnesses.

Q29. Does HAIEC retain customer source code?

The strongest accurate answer is:

HAIEC needs temporary authorized source access for source analysis. The normal workflow uses GitHub account connection plus repository-scoped GitHub App authorization. Source is processed transiently in an ephemeral analysis environment and discarded after analysis. HAIEC retains structured evidence, findings, hashes, metadata and assurance results, not a persisted copy of the repository.

Avoid: "HAIEC never sees code."

Use: "Source is processed, not retained."

Q30. How does GitHub access work?

OAuth links the user's GitHub account to HAIEC. Repository access is handled through the GitHub App and installation-scoped authorization. Customers authorize the

repositories/installations HAIEC may evaluate instead of handing over a long-lived personal token.

Q31. Can HAIEC run in a private environment?

Current public positioning: HAIEC operates as a SaaS with authorized repository access, transient analysis and evidence-only retention.

Enterprise PoC positioning: For organizations that cannot allow external source processing, HAIEC can scope a customer-controlled or private analysis deployment path as part of a design-partner engagement.

Do not claim a fully productized on-prem or self-hosted edition unless that package exists and is validated.

Q32. How secure is HAIEC itself?

Answer with concrete controls and the expected enterprise review path:

- authorized GitHub access,
- source not persisted after analysis,
- ephemeral analysis environment,
- structured evidence retained rather than repository copies,
- transport encryption and encrypted infrastructure where documented,
- no customer data used to train machine learning models.

For regulated enterprise deployment, expect security review, subprocessor review, DPA, access-control review, source-retention review and deployment-model requirements.

Q33. How long does analysis take and what languages are supported?

Only use measured values. Do not estimate scan time, supported LOC scale or language/framework coverage without a verified benchmark.

Current safe answer:

“Performance depends on repository size and enabled analysis depth. HAIEC should report measured analysis runtime and coverage per evaluation. Supported language/framework claims should be limited to the analyzers that are actually production-qualified.”

Q34. Why not combine Semgrep, IAM, LangSmith and a graph database?

Those tools can each provide useful evidence, but they do not make the AI action path the primary assurance object.

HAIEC's unit of assurance is the source-backed route from AI-facing capability to consequence, reconciled against intent, policy, effective authority and observation, then carried forward as a bounded release decision and portable evidence record.

Q35. What business outcome does the enterprise get?

Three outcomes are easiest to defend:

1. Faster release review because the team does not manually reconstruct agent → tool → code → consequence paths.
2. Better change visibility because Consequence Delta shows how the AI-reachable action surface changes between qualified releases.
3. Reusable assurance evidence because one evaluated evidence base can serve engineering, security, governance, release and audit audiences.

Do not invent dollar savings or percentage reductions until measured.